a=3 b=5

2	4	
1	3	✓ in loop - 2 4
0	2	✓ " -1 3
1	1	✓ " 0 2
2	0	✗ " 1 1
		after. 2 0

if

if.. else

while

do.. while

goto

break

continue

Switch.. case

for

✓ else if ladder

~~Comma~~

a	b
3	5
2 &&	4
1 &&	3
0 &&	X

do {

} while(.));

a	b	
3	< 5	
2	< 4	✓ in loop a=-3 b=4
1	< 3	✓ in loop a=-2 b=4
0	< 2	✓ in a=-1 b=3
1	< 1	✓ in a=0 b=2
2	0	✓ in loop a=1 b=1

after loop a=2 b=0

in loop a=3 b=5

" a=-2 b=4

" a=-1 b=3

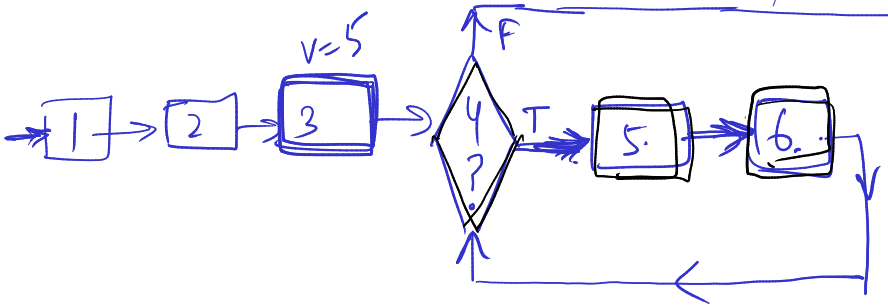
After loop a=0 b=3

the for loop construct

Entry controlled loop

```
for ( initialisation of control var ; test condition ; operation on control variable )
{
Body of loop
}
```

```
1) int v = 1;
2) ++v;
for ( v=5 ; v<=10 ; v++ )
5) Pf("\n loop v=%d\n", v);
7) Pf("After loop v=%d \n", v);
```




```
v=5;
for ( ; v<=10 ; )
{
  Pf("___");
  v++;
}
```

two semicolons MUST in for

```
for ( ; ; ) // Infinite loop
{
  Pf("\n loop \n");
}
```

```
while()
{
  Pf("\n loop \n");
}
```

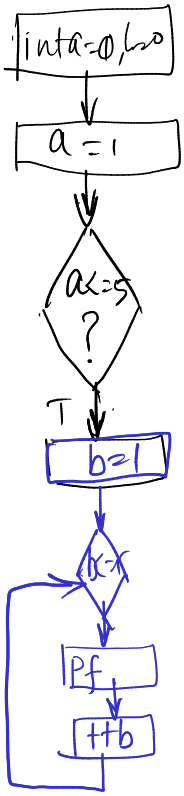
```
for (bit=31, cnt=0; bit>=0; bit--)
{
    if ((data >>bit) &1) cnt++;
}
```

→ if + goto ✓
→ while
→ do ... while
→ for

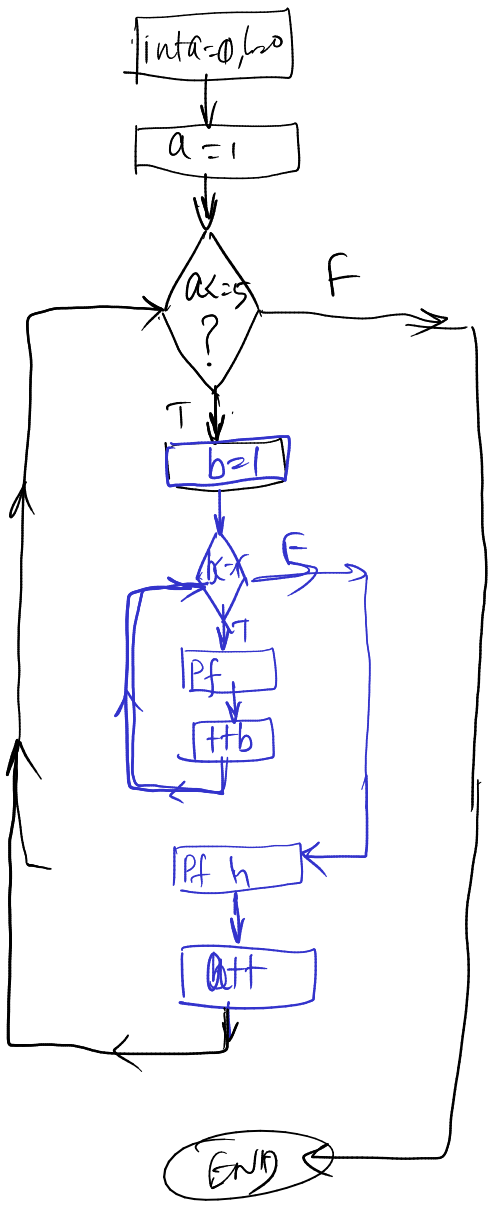
Any C prog., requiring iterations, can be solved by any loop construct.

A common advantage of for loop:

As the loop control stmts present in same line, analysing/reading becomes easier.



int n=45;



Pf("%dn", n);

Pf("%3dn", n); width of printing as 3

Pf("%-5dn", n); with Right Alignment

set width as 5 by default

$:/a.out

45

 4 5

   4 5

Pf("%3d", a*b).

data = 345

34

3

0

max = 56 ⇒ temp

56

5

0

temp->max w=0;

while(temp!=0) {

w++;

temp/=10;

3

w+=2;

data = data/10

34

3

0

logic

Syntax